



Architecture

1 message

Springboardmentor <springboardmentor553@gmail.com> Fri, 6 Mar, 2026 at 07:50

To: rajat roy <seriousargenti@gmail.com>

this a high-level architectural overview of the complete workflow so that you can clearly understand how all the components fit together.

1. Overall Idea of the System

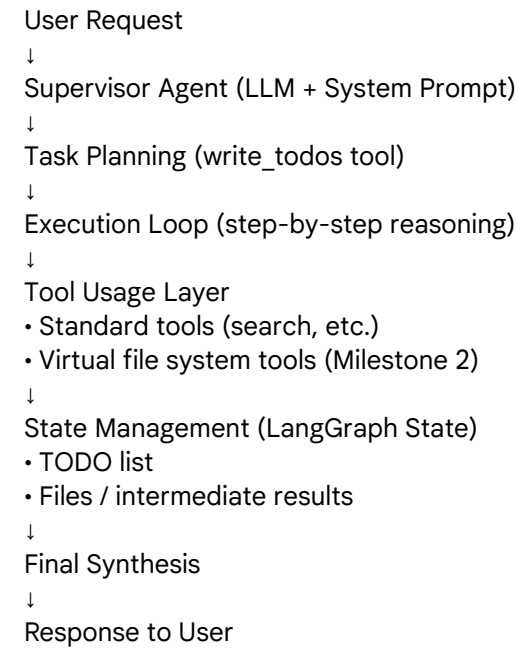
Our project is building an autonomous agent that can handle long-horizon tasks. Instead of directly answering a request in one step, the agent performs the following structured process:

- 1. Understand the user request
- 2. Break the request into smaller tasks
- 3. Execute those tasks step-by-step
- 4. Store intermediate results when needed
- 5. Retrieve stored information later
- 6. Produce the final result

This structured workflow is what allows the agent to handle complex problems without exceeding the model's context window.

2. High-Level Architecture

At a high level, the system consists of the following components:



The LangGraph state acts as the central memory that stores both the TODO list and all intermediate outputs.

3. Milestone 1 – Task Planning Layer

In Milestone 1, the focus was only on planning.

When the user gives a complex task, the agent does not execute it immediately. Instead, it calls the `write_todos` tool to break the problem into smaller steps.

Example:

User request:

“Generate a research report on renewable energy policies.”

The agent creates a TODO list such as:

1. Identify key renewable energy policies globally
2. Collect examples from major countries
3. Summarize policy differences
4. Analyze advantages and limitations
5. Produce final report

These tasks are stored in the LangGraph state under the `todos` list.

At this stage, the agent only creates the plan and does not yet handle large intermediate data.

4. Milestone 2 – Context Offloading via Virtual File System

Milestone 2 introduces external memory management.

When tasks involve large intermediate results (for example long summaries or analysis outputs), storing everything in the conversation context becomes inefficient.

To solve this, we introduced a virtual file system inside the agent’s state.

The agent now has access to four tools:

- `write_file` – store intermediate results
- `read_file` – retrieve stored content
- `edit_file` – modify existing files
- `ls` – list available stored files

Example workflow:

1. The agent completes a research step
2. It stores the result using `write_file("policy_summary.txt", content)`
3. The content is saved inside the state’s file storage
4. Later, when needed, the agent retrieves it using `read_file("policy_summary.txt")`

This allows the agent to manage large tasks while keeping the context window small.

5. Combined Workflow (Milestone 1 + Milestone 2)

When both milestones work together, the complete workflow becomes:

User request received

↓

Agent analyzes request

↓

Agent creates TODO plan (`write_todos`)

↓

Execution loop begins

↓

For each TODO task:

- Agent reasons about next action
- Executes tools if required
- Stores intermediate outputs using `write_file`
 - ↓
 - Later steps retrieve stored information using `read_file`
 - ↓
 - Agent gathers all required results
 - ↓
 - Final synthesis step generates the final output

6. Why This Architecture Matters

This architecture allows the agent to:

- Handle long multi-step tasks
- Avoid exceeding the LLM context window
- Maintain structured memory
- Execute tasks incrementally and reliably

This is the core idea behind building deep research agents and long-horizon autonomous systems.